```
>>> python course
week_01_to_12 = True
for week in learning:
    build_project(week)
```

STRUCTURED • PRACTICAL • PROJECT-DRIVEN

PYTHON PROGRAMMING

Complete Scheme of Work

3 Months / 12 Weeks / 36 Class Sessions

Combines a strong Python foundation with three-part weekly delivery, practical exercises, mini-projects, and a supervised final capstone.

01
FOUNDATIONS

02
CORE PYTHON

03
PRACTICAL BUILD

CURRICULUM • WEEKLY TARGETS • PRACTICAL PROJECTS • ASSESSMENT

CURRICULUM OVERVIEW

Twelve weeks. One clear pathway.

Core knowledge first; active practice every week.

COURSE PARAMETERS

12	3	12	1
TEACHING WEEKS	CLASSES PER WEEK	WEEKLY DELIVERABLES	FINAL CAPSTONE

MONTH-BY-MONTH SCOPE

01 — FOUNDATIONS

WEEKS 1-4

Python setup, variables, conditions, loops, strings and core collections.

02 — CORE PYTHON

WEEKS 5-8

Functions, modules, files, CSV, exceptions, OOP, JSON, APIs and packages.

03 — PRACTICAL BUILD

WEEKS 9-12

Chosen GUI or web pathway, integration, Git/testing and assessed capstone.

THE REPEATABLE 3-CLASS WEEK

01 INTRODUCE

Concepts, syntax breakdown and live coding.

02 PRACTICE

Guided exercises and start mini-project.

03 COMPLETE

Debug, demonstrate, reflect and assess.

Scope principle: The course teaches core Python in sequence. Tkinter OR Flask is a guided application pathway, while web scraping, data analysis and game development are optional exploration labs—not six compulsory specializations.

Python Foundations & CLI

Set up a working Python environment and write simple interactive command-line programs.

KEY LEARNING TOPICS

- What programming is and where Python is used
- Python installation, interpreter and VS Code
- Running .py files; REPL vs script
- Syntax, indentation and comments
- Variables and meaningful naming rules
- Integers, floats, strings and booleans
- The print() function and escape characters
- The input() function
- Arithmetic: +, -, *, /, //, % and **
- Simple string formatting and f-strings
- Combining numeric input with calculations
- Checking types with type()
- Converting input with int() / float()
- Reading basic error messages
- Saving and running a CLI program
- Readable output and basic style

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Install tools, explain scripts and variables, demonstrate print(), input() and common types.

CLASS 2 · GUIDED PRACTICE

Practice user input, arithmetic, conversion, formatting, and start the bill splitter.

CLASS 3 · BUILD + SHOWCASE

Finish, test with different amounts, fix mistakes, and explain each calculation.

WEEKLY PROJECT

Interactive Tip & Bill Splitter

Required deliverable: Accept bill total, tip percentage and number of people; show the tip, full bill and amount per person.

Optional extension: Validate that the number of people is greater than zero.

- **Evidence of learning:** Student runs a script independently and explains a variable, data type and conversion.

Operators, Conditions & Decisions

Use comparisons and Boolean logic to make programs respond to different situations.

KEY LEARNING TOPICS

- Revisiting input and numeric conversion
- Arithmetic and assignment operators
- Comparison operators (==, !=, <, >, <=, >=)
- Boolean expressions and values
- Logical operators: and, or, not
- Operator precedence
- The if statement
- Using elif and else
- Nested conditions
- Indentation and code blocks
- Comparing strings and numbers
- Checking input ranges
- Membership and identity at a basic level
- Simple validation with conditions
- Planning decisions using flowcharts
- Testing boundary cases

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Teach comparisons, Boolean logic and complete if / elif / else demonstrations.

CLASS 2 · GUIDED PRACTICE

Write age checks, eligibility rules and grade boundaries; begin the grade checker.

CLASS 3 · BUILD + SHOWCASE

Finish grading rules, test pass/fail and boundary scores, explain decision paths.

WEEKLY PROJECT

Student Grade Checker

Required deliverable: Collect a student name and score; assign a grade, show pass/fail status, and reject scores outside 0–100.

Optional extension: Add multiple subjects and compute an overall average.

- **Evidence of learning:** Student predicts output for several conditions and explains the difference between = and ==.

Loops, Iteration & Simple Validation

Repeat tasks effectively and control program execution without unnecessary duplication.

KEY LEARNING TOPICS

- Why loops are useful
- Using for with range()
- Range start, stop and step
- Understanding while loops
- Loop conditions and counters
- Break and continue
- The pass statement
- Nested loops and simple patterns
- Iterating through text
- Counters and accumulators
- Avoiding unintended infinite loops
- Basic try/except for invalid input
- Random values using random.randint()
- Input retry loops
- Choosing between for and while
- Simple trace tables and debugging

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Demonstrate for, while, range(), counters, break and continue.

CLASS 2 · GUIDED PRACTICE

Build input retry loops and import random; implement core guessing logic.

CLASS 3 · BUILD + SHOWCASE

Add attempt limits, winning and losing messages, score tracking and debugging.

WEEKLY PROJECT

Guess the Number Game

Required deliverable: Generate a random number, accept guesses, provide hints, and limit attempts with a result summary.

Optional extension: Offer replay without restarting the program.

- **Evidence of learning:** Student chooses an appropriate loop and can trace the number of iterations.

Strings & Data Structures

Store, organize and retrieve collections of information using core Python structures.

KEY LEARNING TOPICS

- String indexing and slicing
- Useful string methods
- Lists: create, access, modify and slice
- append(), remove(), pop() and sort()
- Iterating through lists
- Tuples and immutability
- Sets and unique values
- Set membership and simple operations
- Dictionaries and key/value pairs
- get(), keys(), values() and items()
- Adding, editing and deleting records
- Nested dictionaries and lists
- Choosing an appropriate structure
- Basic list comprehensions (intro only)
- Searching collection contents
- Working with user-entered records

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Teach strings, lists, tuples, sets and dictionaries with comparisons and examples.

CLASS 2 · GUIDED PRACTICE

Practice storing and finding records, adding new scores, and iterating over data.

CLASS 3 · BUILD + SHOWCASE

Finish a menu-driven grade manager and test adding, viewing and searching students.

WEEKLY PROJECT

Student Grade Manager (In Memory)

Required deliverable: Add student scores, list all records, search by student name and compute a simple class average.

Optional extension: Allow one student to have multiple subject scores.

- **Evidence of learning:** Student explains list vs tuple, set uniqueness and dictionary lookup.

Functions, Modules & Reusability

Organize solutions into readable, reusable units and import existing functionality.

KEY LEARNING TOPICS

- Why functions improve programs
- Defining functions with def
- Calling functions and execution flow
- Parameters vs arguments
- Positional and keyword arguments
- Default parameter values
- Using return correctly
- Difference between print and return
- Local vs global variables
- Scope and simple side effects
- Decomposing a problem into functions
- Built-in functions
- Importing standard-library modules
- Creating a custom module
- Using math, random and datetime
- Basic recursion / lambda (enrichment)

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Explain functions, parameters, arguments, returns, scope and simple imports.

CLASS 2 · GUIDED PRACTICE

Refactor repeated code and design a small menu that calls separate functions.

CLASS 3 · BUILD + SHOWCASE

Complete a modular program; demonstrate clear input, processing and output separation.

WEEKLY PROJECT

Function-Based Calculator or Quiz Engine

Required deliverable: Build reusable functions for calculations or quiz scoring, with a menu and a clear result returned to the caller.

Optional extension: Move helper functions to a separate Python module.

- **Evidence of learning:** Student writes a function from scratch and explains what it returns.

Files, CSV, Exceptions & Debugging

Make application data persistent and handle common failures gracefully.

KEY LEARNING TOPICS

- File paths and file modes
- Opening files and closing resources
- The with statement
- Reading .txt files
- Writing and appending text
- Reading / writing .csv with csv
- Rows, columns and headers
- Introduction to JSON storage
- Using json.load and json.dump
- Syntax, runtime and logic errors
- try, except, else and finally
- Handling ValueError and FileNotFoundError
- Raising meaningful errors
- Debugging with print and breakpoints
- Tracing programs step by step
- Testing unexpected or missing input

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Demonstrate text files, CSV, JSON basics and common exception types.

CLASS 2 · GUIDED PRACTICE

Design expense records and save / reload transactions from a CSV file.

CLASS 3 · BUILD + SHOWCASE

Finish reports and summaries; debug invalid amounts, missing files and empty histories.

WEEKLY PROJECT

Persistent Personal Expense Tracker

Required deliverable: Add dated expenses, save to CSV, reload previous entries and calculate a category or overall total.

Optional extension: Export a monthly summary report.

- **Evidence of learning:** Student shows persistence between runs and fixes an intentionally introduced bug.

Object-Oriented Programming

Model real-world entities using classes, objects, attributes and reusable methods.

KEY LEARNING TOPICS

- Procedural vs object-oriented style
- Classes and creating objects
- Instance attributes
- The __init__() constructor
- The self parameter
- Instance methods
- Object state and behavior
- Encapsulation as a design principle
- Basic inheritance
- Method overriding
- Polymorphism through common interfaces
- Composition vs inheritance (overview)
- Class and static methods (overview)
- Reading existing class-based code
- Designing a small object model
- Refactoring procedural code to classes

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Introduce class, object, __init__, self and instance methods with short examples.

CLASS 2 · GUIDED PRACTICE

Model players/items or students/courses and define state-changing methods.

CLASS 3 · BUILD + SHOWCASE

Test the object model, correct class design issues and demonstrate interactions.

WEEKLY PROJECT

Text-Based Adventure or Library Manager

Required deliverable: Use at least two classes with attributes and methods; manage simple interactions or borrowing rules.

Optional extension: Introduce one meaningful inheritance relationship.

- **Evidence of learning:** Student creates an object and explains how its state changes after a method call.

JSON, APIs & External Libraries

Install packages and use structured data returned by a public web API.

KEY LEARNING TOPICS

- Standard library vs third-party packages
- Installing packages with pip
- Virtual environments: create and activate
- Requirements and dependency basics
- JSON structure and common types
- Python dicts vs JSON text
- json.loads, dumps, load and dump
- What an API is
- Client/server request-response model
- HTTP GET and response status codes
- The requests library
- Accessing JSON response data
- Handling request timeouts
- Handling absent keys / malformed input
- API terms, limits and safe use
- Turning responses into readable output

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Explain package installation, JSON parsing and HTTP/API fundamentals.

CLASS 2 · GUIDED PRACTICE

Use requests with an instructor-selected public API; display selected fields.

CLASS 3 · BUILD + SHOWCASE

Finish the data lookup app; handle network errors and invalid requests.

WEEKLY PROJECT

Weather or Currency Information CLI

Required deliverable: Query a public API, parse JSON and display a clear formatted result; include a helpful offline/error message.

Optional extension: Cache the most recent successful response locally.

- **Evidence of learning:** Student identifies request, response, JSON field and external package.

Choose an Application Pathway

Apply Python fundamentals to one beginner-friendly interface pathway without overloading the core course.

KEY LEARNING TOPICS

- Choosing a pathway based on learner goals
- Separating application logic from interface
- Path A: tkinter setup and windows
- Path A: labels, buttons and Entry widgets
- Path A: callbacks and event-driven actions
- Path A: grid / pack layout basics
- Path A: validating widget input
- Path A: updating visible results
- Path B: Flask installation and setup
- Path B: app object and development server
- Path B: routes and URL paths
- Path B: returning HTML responses
- Path B: templates at an introductory level
- Path B: browser/server interaction
- Basic UX and user-facing error messages
- Organizing a small project folder

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Demonstrate both choices, then select tkinter (desktop) OR Flask (web).

CLASS 2 · GUIDED PRACTICE

Build a minimum working interface in the chosen pathway and reuse earlier functions.

CLASS 3 · BUILD + SHOWCASE

Complete a first interface; test interactions and describe the app workflow.

WEEKLY PROJECT

Desktop Calculator OR Personal Profile Website

Required deliverable: Tkinter: working input/display/buttons. Flask: basic routes and pages showing dynamic profile content.

Optional extension: Add two more useful interactions, not merely visual decoration.

- **Evidence of learning:** Student demonstrates an interface with Python-backed behavior.

Extend, Integrate & Polish Your App

Develop one chosen application further while sampling adjacent areas only when time allows.

KEY LEARNING TOPICS

- Revisiting the Week 9 application
- Separating UI, logic and data layers
- Handling validation and empty input
- Refining error feedback
- Using JSON/CSV for simple persistence
- tkinter: frame and form organization
- tkinter: event callbacks and app state
- Flask: HTML forms and GET / POST
- Flask: reading request.form data
- Flask: simple templates and redirect
- Designing clear user workflows
- Manual acceptance testing
- Optional demo: HTML and BeautifulSoup
- Optional demo: pandas DataFrames
- Optional demo: Matplotlib chart basics
- Ethical/safe use of external websites

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Extend chosen toolkit; demonstrate how input reaches Python logic and storage.

CLASS 2 · GUIDED PRACTICE

Add data entry and persistence to the interface; fix edge cases.

CLASS 3 · BUILD + SHOWCASE

Run end-to-end demos, peer-review usability and finish a stable v2 application.

WEEKLY PROJECT

Enhanced GUI or Flask App

Required deliverable: Add a working form, validations and persisted data to the Week 9 project; demonstrate a full task flow.

Optional extension: Instructor-led exploratory lab: scrape a permitted sample page or chart a small CSV.

- **Evidence of learning:** Student traces a complete user action from input to stored data and displayed result.

Git, GitHub, Testing & Capstone

Use a repeatable development workflow and begin an independently planned final application.

KEY LEARNING TOPICS

- Why version control matters
- Installing / configuring Git
- git init, status, add and commit
- Tracking changes over time
- Branches and simple merges (overview)
- GitHub repositories and remote push
- Cloning and README.md basics
- Useful .gitignore rules
- Manual test plans and edge cases
- assert and introductory unittest
- Writing a small automated test
- Naming, structure and documentation
- Choosing a final application problem
- Writing features and acceptance criteria
- Planning milestones and modules
- Building a minimum viable project

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Demonstrate Git, GitHub, assertions and a small test using a familiar project.

CLASS 2 · GUIDED PRACTICE

Define capstone scope, create repository and implement first core feature.

CLASS 3 · BUILD + SHOWCASE

Review project progress, tests and repository; address blocker bugs.

WEEKLY PROJECT

Capstone Project: Milestone 1

Required deliverable: Submit project plan, repository, working first feature, test checklist and clear build steps.

Optional extension: Add automated tests for two core functions.

- **Evidence of learning:** Student makes a meaningful commit and independently explains the chosen project structure.

Capstone Completion & Assessment

Deliver, demonstrate and defend a working, documented Python application.

KEY LEARNING TOPICS

- Completing planned requirements
- Prioritizing essential features
- Connecting functions and data models
- Applying file/JSON persistence
- Adding effective exception handling
- Reviewing OOP use where suitable
- Testing normal and invalid inputs
- Fixing bugs systematically
- Making output and prompts readable
- Organizing files and dependencies
- Writing a clear README.md
- Documenting installation and usage
- Final GitHub push and version tag
- Preparing a concise presentation
- Demonstrating working features
- Reflecting on limitations and next steps

WEEKLY TEACHING RHYTHM

CLASS 1 · CONCEPT + LIVE CODING

Show best practices for finishing, testing, documentation and demonstrations.

CLASS 2 · GUIDED PRACTICE

Conduct final build and peer tests; fix issues and complete README.

CLASS 3 · BUILD + SHOWCASE

Present capstone, explain code decisions, answer questions and receive assessment.

WEEKLY PROJECT

Final Capstone: Working Python Application

Required deliverable: A complete, usable Python app with documented setup, structured code, input checks and demonstrated results.

Optional extension: Outline a next-version feature or a future advanced specialization.

- **Evidence of learning:** Student can explain key code, respond to a changed requirement and justify testing performed.

PROJECT DELIVERY & ASSESSMENT

What students must produce

A realistic, assessable definition of success after three months.

FINAL CAPSTONE STANDARD

- **WORKING APPLICATION**
  A meaningful CLI, desktop or web application solves a clearly stated problem.
- **READABLE PYTHON**
  Functions, sensible names and suitable data structures organize the solution.
- **RELIABILITY**
  Invalid inputs and likely runtime failures are handled appropriately.
- **DATA & INTEGRATION**
  Uses files/JSON or an API when relevant; no unnecessary complexity is required.
- **PROJECT DISCIPLINE**
  README explains setup and usage; Git/GitHub history is available where possible.
- **PRESENTATION**
  Student demonstrates features, explains decisions and answers implementation questions.

ASSESSMENT WEIGHTS

Weekly assignments	**20%**
Practical exercises / project demonstrations	**20%**
Monthly assessments	**20%**
Final capstone application	**30%**
Capstone presentation	**10%**
TOTAL	**100%**

PROGRAMME DESIGN NOTES

Breadth without rushing

Use a single taught application pathway; treat other areas as short tasters or post-course modules.

TWO CORE APPLICATION CHOICES · WEEKS 9–10

TKINTER

DESKTOP GUI

Windows, Entry widgets, buttons, event callbacks and displaying results.

Project: Calculator or form-based records app.

Choose one pathway per class/cohort.

FLASK

BEGINNER WEB

Web routes, templates, simple HTML forms and GET / POST processing.

Project: Profile, feedback or note-taking web app.

Choose one pathway per class/cohort.

ENRICHMENT TASTERS · OPTIONAL / FOLLOW-ON

WEB SCRAPING

HTML structure, requests and BeautifulSoup. Practice only on instructor-approved pages where access and site terms permit.

DATA ANALYSIS

Pandas to read/filter CSV data; Matplotlib for one simple plot using the expense tracker dataset.

GAME DEVELOPMENT

Extend a class-based text adventure with score, player state and turns; a graphical engine is not required.

Design choice: Depth in fundamentals + consistent projects + one practical specialization.

COURSE COMPLETION

Learning outcomes & projects

A practical standard for beginner-to-intermediate progression.

BY THE END, A LEARNER CAN...

- Install Python and independently run scripts in an editor.
- Use variables, data types, expressions, input/output and formatted strings.
- Write conditions, loops and useful validation rules.
- Choose among strings, lists, tuples, sets and dictionaries.
- Design reusable functions and import or create simple modules.
- Read/write files and structured data; handle errors and debug.
- Create and use classes, attributes, constructors and methods.
- Use packages, JSON and a basic public HTTP API.
- Build a small tkinter or Flask application in a guided pathway.
- Use Git basics, write a simple test and document a project.
- Complete and explain a working final Python application.

CAPSTONE PROJECT OPTIONS

Student management system	**To-do list**
Expense tracker	**Inventory tracker**
Quiz or flashcard app	**Weather dashboard (API)**
Contact book	**GUI calculator**
Library manager	**Feedback / notes web app**

Teaching assumption: Three lessons each week, with extra independent practice between classes.